

MergeSafeServer

Control Platform

Software Requirements Document

Comprehensive Design Reference including UI, Database, Modules & Interactions

Document Version:	1.0
Classification:	Internal Use Only Technical Design
Platform:	React + Base44 BaaS
Generated:	May 20, 2026

1. System Overview & Architecture 3

2. Application Layout & Navigation 4

3. Module: Dashboard (System Overview) 5

4. Module: Highways / Settings 6

5. Module: Merge Zones 7

6. Module: Switch Servers 8

7. Module: Sensor Devices 9

8. Module: Triangulation Configuration 10

9. Module: Vehicle Events 11

10. Module: 3D Traffic Visualization 12

11. Module: Data Input Formats 13

12. Module: Users & Access Control 16

13. Module: Portal / Operator Login 17

14. Database Schema & All Entities 18

15. Backend Functions 22

16. Sequence Diagrams 23

17. UI Design System 26

18. Security & Access Control 27

1. SYSTEM OVERVIEW & ARCHITECTURE

MergeSafeServer is a web-based autonomous merge zone control platform for interstate highway management. It provides real-time monitoring, configuration, and telemetry analysis for sensor networks, switch servers, and vehicle detection systems deployed along highway corridors.

1.1 Technology Stack		
Layer	Technology	Purpose
Frontend	React 18 + Vite	Single-page application, component-based UI
Styling	Tailwind CSS + shadcn/ui	Design tokens, utility classes, accessible components
State	@tanstack/react-query	Server state caching and mutations
Routing	react-router-dom v6	Client-side routing with nested layouts
Maps	react-leaflet	Interactive geospatial highway & zone visualisation
3D	three.js	3D traffic scene rendering
Backend/DB	Base44 BaaS	Managed database, auth, serverless functions
Functions	Deno Deploy	Edge serverless functions (TypeScript/JS)
External API	TomTom Traffic API	Real-time traffic flow data for 3D view

1.2 High-Level Architecture	
Browser	React SPA (pages + components)
Browser	Base44 SDK
Browser	Base44 BaaS (entities, auth, db)
Browser	Deno Edge Functions
Browser	External APIs (TomTom)
All entity operations use Base44 SDK methods: list(), filter(), create(), update(), delete()	
Authentication is managed entirely by Base44 platform; no custom auth backend	
Highway-scoped data filtering: a global HighwaySelector persists the selected highway_id throughout all modules	

1.3 Route Map		
Path	Component	Description
/	Dashboard	System overview, stats, map, event feed
/zones	MergeZones	Merge zone CRUD and status
/servers	SwitchServers	Switch server CRUD and health metrics
/sensors	Sensors	Sensor device CRUD and barcode scan
/triangulation	Triangulation	3-switch GPS triangulation config
/events	Events	Vehicle event log with filtering
/traffic3d	Traffic3D	2D map + 3D immersive traffic view
/users	Users	User profile and device assignment management
/input-formats	DataInputFormats	API payload schema configuration
/settings	Settings	Highway configuration + API keys

/portal	Portal	Operator login portal
---------	--------	-----------------------

2. APPLICATION LAYOUT & NAVIGATION

2.1 AppLayout Component

The persistent shell wraps all authenticated routes except /portal. It is composed of a collapsible sidebar and a fixed top header bar.

🔗 Sidebar

- 🔗 Width: 224 px expanded / 64 px collapsed (transition-all duration-300)
- 🔗 Brand logo: circular icon with primary glow + "MergeSafeServer" wordmark
- 🔗 Navigation items rendered from NAV_ITEMS constant (11 routes)
- 🔗 Active state: bg-primary/15 + border-primary/25 + glow-primary class
- 🔗 Collapse toggle button at bottom: ChevronLeft / ChevronRight icon

🔗 Top Header Bar

- 🔗 System Live indicator: pulsing green dot + "SYSTEM LIVE" monospace label
- 🔗 HighwaySelector: dropdown to choose the active highway; propagated via props to all pages
- 🔗 Backdrop blur: bg-card/50 backdrop-blur-sm

2.2 HighwaySelector Component

- 🔗 Fetches Highway entity list on mount via base44.entities.Highway.list()
- 🔗 Renders shadcn/ui Select with highway name + optional icon
- 🔗 Calls onHighwayChange(highway_id) callback to update global state in AuthenticatedApp
- 🔗 Global state held in useState in App.jsx, passed as prop to AppLayout and each page

2.3 PageHeader Component

- 🔗 Props: title (string), subtitle (string, optional), children (action buttons)
- 🔗 Layout: flex row, title/subtitle left, children right
- 🔗 Used uniformly across all module pages

3. MODULE: DASHBOARD (SYSTEM OVERVIEW)

3.1 Purpose

Provides an at-a-glance overview of the entire highway system. Displays key statistics, a geospatial map, and a live event feed. Supports a Demo Mode for simulating traffic without live data.

3.2 UI Fields & Controls

Element	Type	Description
Demo / Exit Demo button	Toggle Button	Activates DemoMap with simulated vehicle events (Zap icon)
Merge Zones stat card	StatCard	Total zones count; red accent if any in fault
Switch Servers stat card	StatCard	online/total ratio; warning if not all online
Sensor Devices stat card	StatCard	online/total ratio
Recent Alerts stat card	StatCard	Count of fault/conflict events in last 20
Map panel	LeafletMap / Three.js	HighwayMap (live) or DemoMap (simulation)
Map legend	Static badges	Zone Active/Fault, Camera/LiDAR/Radar color coding
Merge Zones list	Card list	Links to /zones; shows zone_name, zone_id, mile_marker, StatusBadge
Recent Events / Demo Feed	Scrollable list	Last 20 VehicleEvent records; colour-coded by event_type

3.3 Data Fetching

- ⌚½Parallel Promise.all: MergeZone, SwitchServer, SensorDevice, VehicleEvent (last 20) filtered by selectedHighway
- ⌚½Demo mode: simulated vehicles generated in DemoMap component; events stored in local demoEvents state (max 30)

4. MODULE: SETTINGS / HIGHWAYS

4.1 Purpose

Manages the set of interstate highways in the system and stores external API keys. Highways are the root scope for all other entities.

4.2 UI Fields

Field	Input Type	Validation	Notes
Highway Name	Text input	Required	Human-readable name e.g. Interstate 20
Highway ID	Text input	Required	Unique key used as FK in all child entities
State / Region	Text input	Optional	State abbreviation or name
Description	Text input	Optional	Free-form description
TomTom API Key	Password input	Optional	Saved via saveTomTomKey backend function; show/hide toggle

4.3 Actions

½Add Highway ½ opens Dialog modal with form fields ½ `base44.entities.Highway.create`

½Edit Highway ½ pre-populates form ½ `base44.entities.Highway.update(id, data)`

½Delete Highway ½ `base44.entities.Highway.delete(id)`

½Save TomTom API Key ½ calls `saveTomTomKey` backend function with `{ apiKey }`

5. MODULE: MERGE ZONES

5.1 Purpose

Defines the physical merge zone areas along a highway. A Merge Zone is the root spatial scope for Switch Servers and Sensor Devices. Zones have GPS coordinates and geofence parameters.

5.2 Card Display Fields

zone_name primary title (bold)
zone_id monospace secondary label
StatusBadge coloured badge (active/inactive/fault/maintenance)
mile_marker displayed as "MM {value}"
geofence_radius displayed in metres
latitude / longitude monospace coordinate pair
highway_id highway reference

5.3 Form Fields (Create/Edit Dialog)

Field	Type	Required	Default
Zone Name	Text	Yes	
Zone ID	Text	Yes	
Highway ID	Text	Yes	Pre-filled from selectedHighway
Mile Marker	Number	No	
Latitude	Number	No	
Longitude	Number	No	
Geofence Radius (m)	Number	No	500
Status	Select dropdown	No	active

6. MODULE: SWITCH SERVERS

6.1 Purpose

Manages edge computing nodes (switch servers) that process sensor data and control merge decisions within each zone. Displays health metrics including CPU usage, memory, uptime and network address.

6.2 Card Display Fields

• `server_name` title; `server_id` monospace ID

• `StatusBadge` online / offline / degraded / fault

• IP Address : Port monospace

• CPU / RAM percentage display with animated progress bar

• Uptime formatted as "{h}h {m}m"

• Firmware Version displayed if present

6.3 Form Fields

Field	Input Type	Span	Notes
Server Name	Text	Full width	Required
Server ID	Text	Half	Unique identifier
Zone ID	Text	Half	FK <code>MergeZone.zone_id</code>
Highway ID	Text	Half	FK <code>Highway.highway_id</code>
IP Address	Text	Full width	IPv4 address
Port	Number	Half	Default 8080
Firmware Version	Text	Half	Optional
Status	Select	Half	online/offline/degraded/fault

6.4 Barcode Scanner Integration

• BarcodeScanner component activated via "Scan This Device" dashed button in the dialog

• `Scans JSON` merges all parsed fields into form state

• `Scans plain IP` (4 dot-separated octets) populates `ip_address` only

• `Scans other string` populates `server_id`

7. MODULE: SENSOR DEVICES

7.1 Purpose

Registers and monitors all physical sensor hardware installed along the highway. Supports cameras, LiDAR, radar, vehicle tag readers and signal receivers. Devices are scoped to a Zone and can optionally reference a Switch Server.

7.2 Device Types & Icons

Device Type	Icon	Color Theme
camera	Camera icon	Blue
lidar	Scan icon	Purple
radar	Radio icon	Amber
vehicle tag reader	Tag icon	Green
signal receiver	Antenna icon	Slate

7.3 Form Fields

Field	Type	Notes
Device Name	Text	Required
Device ID	Text	Required, unique identifier
Zone ID	Text	FK !½ MergeZone.zone_id
Highway ID	Text	FK !½ Highway.highway_id
Server ID	Text	Optional FK !½ SwitchServer.server_id
Device Type	Select	camera/lidar/radar/vehicle tag reader/signal receiver
Source Type	Select	physical/satellite/telecom/tracker
Device Height (m)	Number	Installation height
Mile Marker	Number	Position along interstate
Location Comment	Text	Descriptive notes
Latitude / Longitude	Number	GPS coordinates
Status	Select	online/offline/fault/calibrating
Firmware Version	Text	Optional
Notes	Textarea	Free-form notes

8. MODULE: TRIANGULATION CONFIGURATION

8.1 Purpose

Defines three-switch GPS triangulation setups per merge zone. Each configuration records the positions and server references of exactly three switch nodes used for vehicle position triangulation. Used by the Demo Map to show dashed signal-path lines.

8.2 Form Fields

Field	Type	Notes
Zone ID	Text	F K !½ MergeZone.zone_id
Highway ID	Text	F K !½ Highway.highway_id
Geofence Radius (m)	Number	Default 500
Switch N Label	Text (½3)	Display label for each switch (A/B/C)
Switch N Server ID	Text (½3)	F K !½ SwitchServer.server_id
Switch N Latitude	Number (½3)	GPS lat of each switch node
Switch N Longitude	Number (½3)	GPS lon of each switch node
Is Active	Boolean	Default true

8.3 SwitchPanel Sub-component

½Extracted as SwitchPanel({ num, form, setForm }) ½ renders fields for one switch node

½Uses dynamic field prefix: switch{N}_ to access the right fields in the shared form state

½Rendered three times (num=1,2,3) inside the dialog

9. MODULE: VEHICLE EVENTS

9.1 Purpose

Displays a real-time log of sensor-detected vehicle events. Supports filtering by event type and manual refresh. Events are read-only and generated externally or by the Demo simulation.

9.2 Table Columns

Column	Source Field	Format	Width
Type	event_type	UPPERCASE + color +	1/8
Zone	zone_id	Monospace	1/8
Device	device_id	Monospace	1/8
Vehicle	vehicle_id	Monospace	1/8
Speed	speed_mph	{n} mph	1/8
Coordinates	latitude, longitude	5 decimal places	2/8
Time	created_date	HH:mm:ss (date-fns	1/8

9.3 Filter & Controls

🔍 Event type filter Select: all / detection / merge / conflict / speeding / fault

🔄 Refresh button: calls VehicleEvent.filter() again; RefreshCw icon animates during load

📄 Max 100 records loaded, sorted by -created_date

9.4 Event Type Color Coding

Event Type	Color Class	Icon Component
detection	text-primary (blue)	Car
merge	text-accent (green)	Zap
conflict	text-destructive (red)	AlertTriangle
speeding	text-yellow-400	AlertTriangle
fault	text-destructive (red)	AlertTriangle

10. MODULE: 3D TRAFFIC VISUALIZATION

10.1 Purpose

Provides a dual-mode traffic viewer: a 2D Leaflet map landing page with congestion overlays (MapLanding) and a fully immersive Three.js 3D scene (Scene3D) with animated vehicles. Uses TomTom Traffic API for real congestion data.

10.2 Components

- Traffic3D (page) manages mode state (map vs 3D), data fetching, countdown timer (auto-refresh every 5 min)
- MapLanding Leaflet map with road polylines coloured by congestion level; device markers grouped by coordinates
- Scene3D Three.js scene with lane geometry, animated vehicle meshes, directional lighting
- TrafficFeed real-time text feed panel of segment events
- TrafficControls camera angle, speed multiplier, density sliders
- TrafficCanvas2D top-down 2D canvas fallback view

10.3 External API Integration

- getTomTomTraffic backend function fetches /traffic/services/4/flowSegmentData from TomTom
- Returns speed, freeFlowSpeed, currentTravelTime, freeFlowTravelTime, confidence per segment
- If TOMTOM_API_KEY not set, function returns simulated demo data
- Invoked from frontend: `base44.functions.invoke("getTomTomTraffic", { highway_id })`

11. MODULE: DATA INPUT FORMATS

11.1 Purpose

Allows operators to define and manage JSON payload schemas for each of the four data source types. Provides tools to generate synthetic sample payloads, validate them, save them to the database, and duplicate configurations across tabs.

11.2 Tab Structure

- Four tabs: Physical | Satellite | Telecom | Tracker
- Each tab renders an independent FormatTab component with its own state
- Source type colour coding: Physical=Blue, Satellite=Purple, Telecom=Green, Tracker=Orange

11.3 Format Config Card Fields (Display)

Element	Source Field	Style
Format Name	format_name	Bold 0.875rem
Source ID	source_id	Monospace 0.625rem, muted
Input Source	input_source	Monospace 0.625rem, blue-400, !important prefix
Description	description	0.75rem muted
Enabled Field Badges	enabled_fields[0..3]	Source-coloured monospace chips (max 4 + overflow count)
Custom Field Count	custom_fields.length	"{n} custom" chip
Generate Payload btn	•	Full-width outline sm button with Play icon

11.4 Create / Edit Dialog Fields

Field	Input	Required	Notes
Format Name	Text	Yes	Placeholder: "Standard Satellite Feed v1"
Source ID	Text (monospace)	No	Placeholder: "SRC-SAT-001"
Description	Text	No	Free-form description
Input Source	Text	No	Endpoint URL, stream name, or device tag
Common Fields	Checkbox grid	No	20 predefined fields, scrollable max-h-48
Custom Fields	Dynamic rows	No	name=Text, type=Select (string/number/boolean/array/object)

11.5 20 Predefined Common Fields

- make • model • date • time • timestamp • coordinates • latitude • longitude • speed • speed_mph • direction_degrees • firmware_version • status • vehicle_id • tag_id • signal_strength_dbm • vehicle_type • traffic_density_percent • average_speed_mph

11.6 Sample Payload Generation

- Click "Generate Sample Payload" on a config card
- Calls generateInputPayload backend function with { source_type, enabled_fields, custom_fields }
- Payload preview panel shows: validation status (CheckCircle/AlertCircle), Copy button, Save button, close (X) button
- Save button • creates SamplePayload entity record; payload stored as JSON string

11.7 Saved Payloads Section

- Displayed below the preview panel when `savedPayloads.length > 0`
- Each saved payload row: validity icon, label/format_name, created timestamp, copy/duplicate/delete actions
- Payload JSON displayed in expandable monospace green-on-dark code block (max-h-32)
- Duplicate: creates new SamplePayload with label "(Copy)"

11.8 Cross-Tab Format Duplication

- Toolbar "Duplicate" dropdown: shows all configs across all source types
- User selects a source format and a target source type, then clicks Duplicate
- Creates a new InputFormatConfig with `source_type = selected target`, `format_name += "(Copy)"`

12. MODULE: USERS & ACCESS CONTROL

12.1 Purpose

Manages operator and viewer user profiles stored in the UserProfile entity. Each profile can be associated with a specific highway and granted access to specific sensor devices. This is an application-level profile layer separate from Base44 authentication.

12.2 Table Columns

Column	Source	Visibility
User (avatar + name + ID)	full_name, user_id	Always
Phone	phone	md+
Address	address	lg+
Type	user_type	Always (colour badge)
Interstate	highway_name	md+
Devices	device_ids.length	lg+
Status	is_active	Always (green pulse / grey)
Actions	Edit / Delete icons	Always

12.3 User Type Color Badges

Type	Background	Text
admin	bg-red-500/15	text-red-400
operator	bg-blue-500/15	text-blue-400
viewer	bg-slate-500/15	text-slate-400
technician	bg-yellow-500/15	text-yellow-400
supervisor	bg-purple-500/15	text-purple-400

12.4 Add / Edit Dialog Fields

Field	Input	Notes
Full Name	Text	Required
User ID	Text (monospace)	e.g. USR-0042
Phone	Text	Free-form
User Type	Select	admin/operator/viewer/technician/supervisor
Address	Text (full width)	Street address
Interstate Assignment	Select dropdown	From Highway entity list; resets device selection
Device Access (search)	Text search	Filters devices by name/ID/type/mile_marker
Device Checkboxes	Custom checkbox	Scrollable 14rem; toggle adds/removes from device_ids & device_names arrays
Notes	Text	Optional free-form

Status	Select	active / inactive ¹ / ₂ stored as is_active boolean
--------	--------	---

13. MODULE: PORTAL / OPERATOR LOGIN

13.1 Purpose

A standalone login page at /portal accessible outside the AppLayout shell. Allows highway operators to authenticate against their assigned UserProfile records and enter a monitoring session scoped to their highway.

13.2 UI Fields

Field	Type	Notes
Interstate selector	Select dropdown	Loads from Highway entity; required
Password / Access Code	Password input	Show/hide toggle via Eye/EyeOff icon
Login button	Submit button	Disabled while loading; shows spinner

13.3 Authentication Flow

- On submit: validates highway_id and password are filled
- Calls `base44.entities.UserProfile.filter({ highway_id })` to get profiles for that highway
- Checks password match against profile records (demo: any non-empty value accepted)
- On success: stores highway_id in `sessionStorage` ! redirects to / (dashboard)
- Existing session detected on mount ! immediate redirect to /

14. DATABASE SCHEMA 14.1 ALL ENTITIES

All entities are managed by the Base44 BaaS platform. Every record has built-in fields: id (string), created_date (ISO datetime), updated_date (ISO datetime), created_by (email string).

14.1 Highway

Field	Type	Required	Description / Enum Values
name	string	Yes	Human-readable highway name
highway_id	string	Yes	Unique identifier (PK-like)
description	string	No	Optional description
state	string	No	State or region
is_active	boolean	No	Default: true

14.2 MergeZone

Field	Type	Required	Description / Enum Values
highway_id	string	Yes	F K 14.2 Highway . highway_id
zone_name	string	Yes	Display name
zone_id	string	Yes	Unique zone identifier
mile_marker	number	No	Position along the interstate
latitude / longitude	number	No	GPS coordinates
status	enum	No	active inactive fault maintenance
geofence_radius	number	No	Metres, default 500

14.3 SwitchServer

Field	Type	Required	Description / Enum Values
zone_id	string	Yes	F K 14.3 MergeZone . zone_id
highway_id	string	No	Denormalised highway reference
server_name	string	Yes	Display name
server_id	string	Yes	Unique server identifier
ip_address	string	No	IPv4 address
port	number	No	Default 8080
status	enum	No	online offline degraded fault
cpu_percent	number	No	Default 0
memory_percent	number	No	Default 0
uptime_seconds	number	No	Default 0
last_heartbeat	datetime	No	ISO 8601
firmware_version	string	No	Semver string

14.4 SensorDevice

Field	Type	Required	Description / Enum Values
zone_id	string	Yes	F K !½ MergeZone.zone_id
highway_id	string	No	Denormalised highway reference
server_id	string	No	F K !½ SwitchServer.server_id
device_name	string	Yes	Display name
device_id	string	Yes	Unique device identifier
device_type	enum	Yes	camera lidar radar vehicle tag reader signal receiver
source_type	enum	No	physical satellite telecom tracker
device_height	number	No	Installation height in metres
mile_marker	number	No	Position along the interstate
location_comment	string	No	Descriptive location note
latitude / longitude	number	No	GPS coordinates
status	enum	No	online offline fault calibrating
last_seen	datetime	No	ISO 8601 timestamp
firmware_version	string	No	Semver string
notes	string	No	Free-form notes

14.5 TriangulationConfig

Field	Type	Required	Description / Enum Values
zone_id	string	Yes	F K !½ MergeZone.zone_id
highway_id	string	No	Denormalised reference
geofence_radius	number	No	Default 500
switch{1-3}_server_id	string	No	F K !½ SwitchServer.server_id (½3)
switch{1-3}_lat	number	No	GPS latitude of switch node (½3)
switch{1-3}_lon	number	No	GPS longitude of switch node (½3)
switch{1-3}_label	string	No	Display label (½3)
is_active	boolean	No	Default true

14.6 VehicleEvent

Field	Type	Required	Description / Enum Values
zone_id	string	Yes	F K !½ MergeZone.zone_id
highway_id	string	No	Denormalised reference
server_id	string	No	Source server
device_id	string	No	Source device

event_type	enum	Yes	detection merge conflict speeding fault
vehicle_id	string	No	Detected vehicle identifier
speed_mph	number	No	Vehicle speed
latitude / longitude	number	No	GPS location at event time
confidence	number	No	Detection confidence 0 to 100
timestamp	datetime	No	Event timestamp (ISO 8601)
raw_data	string	No	JSON payload from sensor

14.7 InputFormatConfig

Field	Type	Required	Description / Enum Values
source_type	enum	Yes	physical satellite telecom tracker
format_name	string	Yes	User-friendly name for the format
source_id	string	No	Unique input source identifier
input_source	string	No	Endpoint URL, stream name, or device tag
description	string	No	Free-form description
enabled_fields	string[]	No	Array of predefined field keys
custom_fields	object[]	No	Array of { name: string, type: string }

14.8 SamplePayload

Field	Type	Required	Description / Enum Values
source_type	enum	Yes	physical satellite telecom tracker
format_config_id	string	Yes	FK to InputFormatConfig.id
format_name	string	No	Snapshot of format_name at save time
payload	string	Yes	JSON-stringified generated payload
is_valid	boolean	No	Validation result flag
label	string	No	Optional user label (used for copies)

14.9 UserProfile

Field	Type	Required	Description / Enum Values
user_id	string	No	Application-level user identifier
full_name	string	Yes	Display name
address	string	No	Street address
phone	string	No	Contact phone
user_type	enum	No	admin operator viewer technician supervisor
highway_id	string	No	FK to Highway.highway_id
highway_name	string	No	Denormalised highway name
device_ids	string[]	No	Array of device_id values the user can access
device_names	string[]	No	Parallel array of display names
notes	string	No	Free-form notes
is_active	boolean	No	Default true

14.10 Entity Relationships

1 Highway (1) % % % % < MergeZone (highway_id)
1 Highway (1) % % % % < SwitchServer (highway_id)
1 Highway (1) % % % % < SensorDevice (highway_id)
1 Highway (1) % % % % < UserProfile (highway_id)
1 MergeZone (1) % % % % < SwitchServer (zone_id)
1 MergeZone (1) % % % % < SensorDevice (zone_id)
1 MergeZone (1) % % % % < VehicleEvent (zone_id)
1 MergeZone (1) % % % % < TriangulationConfig (zone_id)
1 SwitchServer (1) % % % % < SensorDevice (server_id, optional)
1 InputFormatConfig (1) % % % % < SamplePayload (format_config_id)
1 UserProfile.device_ids[] % % % % > < SensorDevice.device_id (M:N via array)

15. BACKEND FUNCTIONS

Serverless edge functions deployed on Deno Deploy. All functions authenticate the caller via `base44.auth.me()` before executing business logic.

Function	Inputs	Output	Description
<code>generateInputPayload</code>	<code>source_type</code> , <code>enabled_fields[]</code> ,	<code>{ payload, validation,</code>	Generates randomised telemetry JSON from
<code>getTomTomTraffic</code>	<code>highway_id</code>	traffic segment array	Calls TomTom Traffic Flow API. Falls back to
<code>saveTomTomKey</code>	<code>{ apiKey }</code>	<code>{ success }</code>	Persists TomTom API key as

15.1 `generateInputPayload` Logic Detail

• `SOURCE_BASE`: per source-type base fields (`sensor_id`, `tracker_id`, `device_id`, `fleet_id`)

• `COMMON_FIELD_SAMPLES`: 20 field definitions with type and sample values

• `generateUnique()`: random timestamp (±60 min), GPS ±0.1° around Dallas, speed 20±100 mph, random direction 0°±360°

• Builds `payload.data` object by iterating `enabled_fields` and `custom_fields`

• Validation: fails if no fields selected; fails if custom field missing name or type

• Returns full payload JSON + validation summary object

16. SEQUENCE DIAGRAMS

16.1 Application Bootstrap

Browser	!!ç½	AuthProvider	Mounts; calls base44.auth.me()
AuthProvider	!!ç½	Base44 BaaS	GET /auth/me (bearer token from cookie)
Base44 BaaS	!!ç½	AuthProvider	Returns user object or 401
AuthProvider	!!ç½	AuthenticatedApp	Sets isLoadingAuth=false, provides user context
AuthenticatedApp	!!ç½	AppLayout	Renders layout with Outlet
AppLayout	!!ç½	HighwaySelector	Mounts; fetches Highway.list()
HighwaySelector	!!ç½	AppLayout	onHighwayChange(highway_id) on selection
AppLayout	!!ç½	Page Component	Passes selectedHighway as prop

16.2 Create / Edit Entity (Generic CRUD)

User	!!ç½	Page	Clicks "Add / Edit" button
Page	!!ç½	Dialog	setOpen(true); form pre-populated if editing
User	!!ç½	Dialog	Fills form fields, clicks Save/Create
Dialog	!!ç½	save()	Calls save() handler
save()	!!ç½	Base44 SDK	Entity.create(data) or Entity.update(id, data)
Base44 SDK	!!ç½	Base44 BaaS	HTTP POST/PUT to database API
Base44 BaaS	!!ç½	Base44 SDK	Returns saved record
save()	!!ç½	Page	setOpen(false); load() refreshes list

16.3 Generate & Save Sample Payload

User	!!ç½	FormatTab	Clicks "Generate Sample Payload" on a config card
FormatTab	!!ç½	generatePayload()	setSelectedConfig(cfg); setGenerating(true)
generatePayload()	!!ç½	base44.functions	invoke("generateInputPayload", { source_type, enabled_fields, custom_fields })
base44.functions	!!ç½	Deno Edge Fn	HTTP POST with payload config
Deno Edge Fn	!!ç½	base44.functions	Returns { payload, validation, field_count }
FormatTab	!!ç½	Payload Preview	setPayload(res.data.payload); renders preview panel
User	!!ç½	FormatTab	Clicks "Save" button in preview panel
FormatTab	!!ç½	savePayload()	setSaving(true)
savePayload()	!!ç½	Base44 SDK	SamplePayload.create({ source_type, format_config_id, payload, is_valid })
Base44 SDK	!!ç½	Base44 BaaS	Persists record to database
savePayload()	!!ç½	FormatTab	setSaving(false); load() refreshes savedPayloads list

16.4 Cross-Tab Format Duplication

User	!!ç½	FormatTab	Opens "Duplicate" dropdown toolbar button
FormatTab	!!ç½	allConfigs	Lists all InputFormatConfig records across all source types
User	!!ç½	DropDownMenu	Selects a source config and a target source type

User	!i½	FormatTab	Clicks "Duplicate" confirm button
duplicateTo()	!i½	Base44 SDK	InputFormatConfig.create({ source_type: target, format_name + "(Copy)", ... })
Base44 SDK	!i½	FormatTab	Returns new config; load() refreshes the target tab data

16.5 Dashboard Data Load with Highway Filter

User	!!_½	HighwaySelector	Selects "Interstate 20"
HighwaySelector	!!_½	AppLayout	onHighwayChange("I20")
AppLayout	!!_½	Dashboard	Re-renders with selectedHighway="I20"
Dashboard useEffect	!!_½	Base44 SDK	Promise.all: MergeZone.filter({highway_id:"I20"}), SwitchServer.filter({...}), SensorDevice.filter({...}), VehicleEvent.filter({...}, -created_date, 20)
Base44 BaaS	!!_½	Dashboard	Returns filtered arrays; setZones/setServers/setSensors/setEvents
Dashboard	!!_½	StatCards	Re-renders with new counts
Dashboard	!!_½	HighwayMap	Passes zones and sensors for map rendering

16.6 TomTom Traffic Data Flow

Traffic3D	!!_½	fetchTraffic()	User clicks refresh or 5-min countdown expires
fetchTraffic()	!!_½	base44.functions	invoke("getTomTomTraffic", { highway_id })
base44.functions	!!_½	Deno Edge Fn	Authenticated HTTP POST
Deno Edge Fn	!!_½	TomTom API	GET /traffic/services/4/flowSegmentData?key=TOMTOM_API_KEY
TomTom API	!!_½	Deno Edge Fn	Returns speed, freeFlowSpeed, travelTime per segment
Deno Edge Fn	!!_½	Traffic3D	Returns structured traffic data array
Traffic3D	!!_½	Scene3D/MapLanding	Passes trafficData prop; components re-render with new congestion colours

16.7 Operator Portal Login

User	!!_½	Portal	Selects highway and enters access code; clicks Login
Portal	!!_½	Base44 SDK	UserProfile.filter({ highway_id: selectedHighway })
Base44 SDK	!!_½	Portal	Returns profile records for that highway
Portal	!!_½	Portal (local)	Validates credentials against returned profiles
Portal	!!_½	sessionStorage	sessionStorage.setItem("highway_id", value)
Portal	!!_½	Browser	window.location.href = "/" (redirect to dashboard)

17. UI DESIGN SYSTEM

17.1 Color Tokens (CSS Variables)

Token	Light Mode	Dark Mode	Usage
--background	hsl(0 0% 100%)	hsl(0 0% 3.9%)	Page background
--foreground	hsl(0 0% 3.9%)	hsl(0 0% 98%)	Primary text
--card	hsl(0 0% 100%)	hsl(0 0% 3.9%)	Card/panel backgrounds
--primary	hsl(0 0% 9%)	hsl(0 0% 98%)	Buttons, active nav, accents
--secondary	hsl(0 0% 96.1%)	hsl(0 0% 14.9%)	Subtle backgrounds, hover
--muted	hsl(0 0% 96.1%)	hsl(0 0% 14.9%)	Muted text, placeholder backgrounds
--border	hsl(0 0% 89.8%)	hsl(0 0% 14.9%)	All borders and dividers
--destructive	hsl(0 84.2% 60.2%)	hsl(0 62.8% 30.6%)	Errors, delete actions

17.2 Typography

Font: system sans-serif (Helvetica / Arial fallback stack via Tailwind default)

Page titles: text-lg font-bold tracking-tight

Section headers in cards: text-sm font-semibold

Body text: text-sm

Metadata / IDs / code: text-xs font-mono text-muted-foreground

Badge / chip text: text-[10px] or text-[11px]

17.3 Component Patterns

Cards: rounded-xl border border-border bg-card p-4 consistent across all module grids

Dialogs: max-w-lg (standard) / max-w-2xl (complex forms like Triangulation, Users)

Buttons: shadcn/ui Button with size="sm" for actions within cards; default for primary dialog actions

Status Badges: StatusBadge shared component maps status string to colour class

Tables: grid-based layout with monospace column data (Events) or HTML table (Users)

Inputs: h-8 text-sm for compact inline forms; default h-9 in dialogs

Source-type colour chips: tab-specific colour for field badges (blue/purple/green/orange)

17.4 Responsive Breakpoints

Mobile: single column card grid, condensed table (some columns hidden)

md (768 px): 2-column card grid; table shows Phone, Interstate columns

xl (1280 px): 3-column card grid

Table responsive: phone/address/devices hidden below md/lg

17.5 Animation & Feedback

Loading spinner: border-t-primary rounded-full animate-spin (8 or custom size)

System Live indicator: animate-pulse on green dot in header

Demo mode event feed: "LIVE" badge with animate-pulse text-yellow-400

CPU progress bar: transition-all on width change

Copy confirmation: "Copied!" text replaces "Copy" for 2 seconds

1Save confirmation: keySaved state shows "Saved!" + CheckCircle icon for 2.5 s

18. SECURITY & ACCESS CONTROL

18.1 Authentication

- Handled entirely by Base44 platform; no custom auth implementation
- Session tokens managed via HTTP-only cookies by Base44 BaaS
- All entity API calls are authenticated via the Base44 SDK token
- Backend functions authenticate caller via `base44.auth.me()` on every request
- Unauthenticated function calls return 401 Unauthorized

18.2 Authorization

- User roles stored in Base44 User entity: admin / user (default)
- UserProfile entity stores application roles: admin, operator, viewer, technician, supervisor
- Admin-only operations protected by `user.role === "admin"` checks in backend functions
- Highway-scoped data: all listing operations filtered by `highway_id` prop
- UserProfile `device_ids` array controls which sensors a user profile has access to (application-level)

18.3 API Keys & Secrets

- TomTom API key stored as `TOMTOM_API_KEY` environment secret (not exposed to frontend)
- `saveTomTomKey` backend function updates the secret; only accessible to authenticated users
- `BASE44_APP_ID` is the only pre-populated secret available in edge functions
- No other API keys or credentials are stored client-side

18.4 Portal Access

- portal route is outside the AppLayout shell • accessible without the main navigation
- Operator login validated against UserProfile records for the selected highway
- Session stored in sessionStorage (tab-scoped, cleared on tab close)
- No sensitive credentials transmitted; password field is a demo access code